Practical 9

Implementation of Prim's algorithm

1. Introduction

Prim's Algorithm is a classic greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph. A Minimum Spanning Tree is a subset of the edges that connects all the vertices together, without any cycles, and with the minimum possible total edge weight.

The algorithm starts from an arbitrary vertex and grows the tree one edge at a time. At every step it adds the cheapest edge that connects a vertex already in the tree to a vertex outside the tree, until all the vertices are included.

2. Aim

To implement Prim's Algorithm in C++ using an adjacency matrix to find the Minimum Spanning Tree of a weighted graph and determine its total weight.

3. Objectives

- To understand the concept of a Minimum Spanning Tree.
- To understand the greedy approach used in Prim's Algorithm.
- To represent a weighted graph using an adjacency matrix.
- To find the edges of the MST and its total weight.
- To detect a disconnected graph.
- To analyze the time and space complexity of the algorithm.
- To implement the algorithm using C++.

4. Algorithm

Step 1: Start.

Step 2: Read the graph as an adjacency matrix of V vertices.

Step 3: Create the arrays key[], parent[] and mst[], where:

- key[i] stores the minimum weight of an edge connecting vertex i to the tree.
- parent[i] stores the vertex through which i is connected to the tree.
- mst[i] is true if vertex i is already included in the tree.

Step 4: Initialize all key values to infinity and all mst values to false.

Step 5: Set key[0] = 0 and parent[0] = −1, so that vertex 0 is picked first.

Step 6: Repeat V − 1 times: pick the vertex u that is not in the MST and has the minimum key value.

Step 7: If no such vertex is found (u = −1), the graph is disconnected; print a message and stop.

Step 8: Include u in the MST by setting mst[u] = true.

Step 9: For every vertex v adjacent to u that is not in the MST, if graph[u][v] < key[v], update key[v] = graph[u][v] and parent[v] = u.

Step 10: After the loop, print the edges (parent[i], i) with their weights and add the weights to get the total MST weight.

Step 11: Stop.

5. Pseudocode

```
BEGIN
FUNCTION Prim(graph, V)
    FOR i ← 0 TO V − 1
        key[i] ← INFINITY, mst[i] ← false
    END FOR
    key[0] ← 0, parent[0] ← −1
    FOR count ← 0 TO V − 2
        u ← vertex not in MST with minimum key[u]
        IF u does not exist THEN
            PRINT "Graph is disconnected" and RETURN
        END IF
        mst[u] ← true
        FOR v ← 0 TO V − 1
            IF graph[u][v] ≠ 0 AND mst[v] = false AND graph[u][v] < key[v] THEN
                parent[v] ← u
                key[v] ← graph[u][v]
            END IF
        END FOR
    END FOR
    total ← 0
    FOR i ← 1 TO V − 1
        PRINT parent[i], i, graph[i][parent[i]]
        total ← total + graph[i][parent[i]]
    END FOR
    PRINT total
END FUNCTION
END
```

Code:

```
#include <iostream>
using namespace std;

#define V 5
#define INF 999999

void prim(int graph[V][V]) {
    int parent[V];
    int key[V];
    bool mst[V] = {false};

    for (int i = 0; i < V; i++)
        key[i] = INF;
```

```
    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int min = INF, u = -1;

        for (int i = 0; i < V; i++) {
            if (!mst[i] && key[i] < min) {
                min = key[i];
                u = i;
            }
        }

        if (u == -1) {
            cout << "Graph is disconnected";
            return;
        }

        mst[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !mst[v]
                && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    int total = 0;
    cout << "Edge \tWeight\n";

    for (int i = 1; i < V; i++) {
        cout << parent[i] << " - "
             << i << "\t" << graph[i][parent[i]] << "\n";
        total += graph[i][parent[i]];
    }

    cout << "Total MST Weight: " << total;
}

int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };

    prim(graph);

    return 0;
}
```

Output:

```
Edge   Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5
Total MST Weight: 16
```

The graph has 5 vertices (0 to 4). Starting from vertex 0, the algorithm picks the edges 0–1 (2), 1–2 (3), 1–4 (5) and 0–3 (6), giving a Minimum Spanning Tree with a total weight of 2 + 3 + 5 + 6 = 16.

6. Advantages

- Always produces a Minimum Spanning Tree for a connected weighted graph.
- Uses a simple greedy approach that is easy to understand and implement.
- Works well for dense graphs when an adjacency matrix is used.
- Grows a single tree, so no cycle detection is required.
- Can detect a disconnected graph.

7. Disadvantages

- The adjacency matrix implementation takes $O(V^2)$ time, which is slow for sparse graphs.
- Requires $O(V^2)$ memory to store the adjacency matrix.
- Works only on connected, undirected, weighted graphs.
- Edges with weight 0 cannot be represented, since 0 is used to mean no edge.
- Not suitable for finding shortest paths.

8. Complexity Analysis

Complexity	**Value**
Time Complexity	$O(V^2)$
Space Complexity	$O(V^2)$

Where:

- V = number of vertices

The space is $O(V^2)$ because of the adjacency matrix; the extra arrays key, parent and mst need only O(V).

9. Conclusion

Prim's Algorithm was successfully implemented in C++ using an adjacency matrix. The algorithm repeatedly selects the minimum weight edge that connects a new vertex to the growing tree, and stores the best connection for each vertex in the key and parent arrays. It produced the Minimum Spanning Tree with a total weight of 16 for the given graph. Thus, Prim's Algorithm provides an efficient and systematic greedy approach for solving the Minimum Spanning Tree problem.